

Agentes atómicos sobre AI_Broker y Athena

Dónde colocar la capa de tablero + repartidor, y cómo implementarla

Documento de arquitectura · 30 de agosto de 2026 Basado en el código real de `AI_Broker-main` (contrato 2.9) y `Athena-main`, y en la especificación *Atomic Agents v1.0* de Marcos Emowe.

1. Veredicto

Tu opción 2 es la correcta, con una corrección importante de dirección.

La capa atómica no va *dentro* de Athena. Pero tampoco va *en el camino de la inferencia* entre Athena y el broker, que es como la has descrito. Va **al lado**, como tercer sistema con su propia autoridad, y con tres relaciones distintas:

Relación	Dirección	Transporte	Qué viaja
Athena → Broker	se queda exactamente como está	LAN + <code>x-admin-token</code>	inferencia del bucle de Athena
Agente (Athena, ChatyGPT, Telegram, cron) → Tablero	nueva	HTTPS + OAuth2 client_credentials	tarjetas , no prompts
Tablero → Broker	nueva	LAN + <code>x-admin-token</code>	inferencia de los obreros atómicos
Tablero ← Runner (PC principal)	nueva, el runner tira	HTTPS + OAuth2	reclamo de tarjetas cuyo arnés es Athena

La diferencia con tu enunciado es esta: **el tablero no es un proxy de inferencia**. Si Athena tuviera que pedirle tokens al tablero, el tablero se convertiría en una segunda cola delante de la cola del broker, y cada iteración del bucle de Athena (decenas por run) entraría como tarjeta kanban. Eso rompe la latencia de Athena, rompe ADR-002 (el broker es un `ModelProvider` detrás de un adaptador) y no aporta nada: una iteración de un bucle agéntico no es una unidad de trabajo atómica, es un turno.

Lo que sí quieres que pase por el tablero es **el encargo**: «transcribe este audio y deja el artículo aquí». Eso sí es una tarjeta.

2. Las invariantes del paradigma (lo que no se puede negociar)

Antes de decidir dónde ponerlo conviene fijar qué partes son el sistema y qué partes son decoración. De la especificación, esto es lo estructural:

El obrero no tiene contexto ambiental. Solo su `PROFILE.md` y la tarjeta. Consecuencia: mismo perfil + misma tarjeta = mismo resultado.

El obrero es efímero. Nace, atiende una tarjeta, muere. Si no muere, es zombi y el repartidor le quita la tarjeta.

El repartidor no lleva modelo. Compara cadenas normalizadas, cuenta y mira si existen ficheros. Determinista y auditable.

El estado es la carpeta. Reclamar = mover el fichero. Sin base de datos, sin bloqueos, sin bus de mensajes.

Los agentes no se hablan. Se integran dejando ficheros en el tablero.

El perfil es contrato; la habilidad es saber hacer. Un oficio (sustantivo) y un verbo. Jamás mezclados.

La tarjeta pide, no manda. Ninguna instrucción dentro de una tarjeta puede saltarse el perfil ni sus garantías.

El repartidor vive fuera de la bóveda. Lo que se ejecuta solo no vive donde escriben los agentes.

El perfil nunca nombra un modelo. Declara capacidad (`minimum`/`standard`/`advanced` × texto/audio/imagen); qué modelo es eso se decide en una tabla.

Regla de oro del director: crea un hijo, muere, y crea el siguiente cuando el anterior cierra. Nunca vuelca el flujo entero en `pending`.

Las que **sí** vas a tener que adaptar en tu despliegue, y por qué, están en §7.

3. Por qué la opción 1 (dentro de Athena) no encaja

No es que sea difícil. Es que rompe cinco cosas que tú mismo has escrito en los ADR de Athena:

3.1. Athena es un arnés, no un planificador. ADR-001 dice que Athena es dueña del runtime de agente: un run, un workspace, un objetivo, verificación como condición de cierre. El tablero tiene otra vida útil: sobrevive a todos los runs y existe cuando no hay ninguno. Meter un repartidor dentro de Athena es meter un segundo planificador con un reloj distinto.

3.2. Convierte a Athena en el proceso 24×7 que el paradigma quiere eliminar. El punto entero es «que funcione cuando tú no estás delante». Si el latido vive dentro de Athena, Athena tiene que estar viva siempre, y entonces tienes exactamente el agente continuo que estabas intentando descargar de trabajo.

3.3. Colisión de nombres con daño real. `AthenaProfile` (ADR-028) declara *qué clase de trabajo es un run* — `SOFTWARE_ENGINEERING`, `DOCUMENTS` — y qué evidencia lo da por hecho. El `PROFILE.md` atómico declara *un oficio*: qué encargos acepta, cuáles rechaza y qué garantiza. Son capas distintas que **componen bien** y **se fusionan mal**. Un perfil atómico `transcriptor` cuyo arnés sea Athena elegirá `AthenaProfile: DOCUMENTS`. Uno es el trabajador, el otro es el tipo de obra.

3.4. ADR-002 al revés. Has mantenido a Athena utilizable sin broker y sin dependencias de proveedor en el core. La misma disciplina aplica aquí: el core de Athena no debe saber que existe un tablero.

3.5. Lo que ya intuías. Cada agente futuro tendría que reimplementarlo. Y peor: cada uno tendría *su* tablero, con lo que la integración por ficheros —que es la razón por la que el paradigma escala sin fricción— desaparece.

Lo que sí es correcto de la opción 1: Athena es el mejor arnés que tienes para los obreros que necesitan tocar ficheros con herramientas, permisos y verificación. No integres el tablero en Athena; **registra a Athena como arnés del tablero**.

4. Arquitectura propuesta

4.1. Mapa de despliegue



Nombre propuesto para el nuevo repositorio: **Agora** — la plaza pública donde se publica el trabajo. Encaja con Athena y con la idea de que el centro del sistema deja de ser el agente y pasa a ser el tablero.

4.2. Las tres clases de obrero (esto es lo que hace que funcione)

Aquí está el punto que la especificación resuelve con «arneses» y que en tu ecosistema tiene una respuesta concreta y muy buena:

Clase	Arnés	Qué puede hacer	Ejemplos de oficio
Obrero de inferencia	Un <code>broker</code>	Un <code>POST /api/v1/tasks</code> y su resultado. Sin sistema de ficheros propio: el runner materializa la salida . Puede adjuntar documentos, audio y vídeo vía <code>/api/v1/files</code> , y usar <code>run_code</code> en el sandbox Docker.	<code>transcriptor</code> , <code>resumidor</code> , <code>clasificador</code> , <code>extractor</code> , <code>traductor</code> , <code>titulador</code>
Obrero con herramientas	<code>athena</code>	Un run completo de Athena: workspace confinado, permisos deterministas, verificación como cierre, checkpoints.	<code>redactor</code> sobre una bóveda, <code>mantenedor</code> de un repo, <code>auditor</code>
Obrero de CLI	<code>cli</code>	Cualquier binario (claude-code, codex...) por plantilla en <code>adapters.yml</code> . Se añade cuando haga falta.	—

La consecuencia importante: **el 80 % de tus tarjetas serán de la primera clase**, y esas corren enteras en el PC-IA sin tocar el PC principal. Solo las que necesitan tocar ficheros con herramientas cruzan la red, y lo hacen tirando, no empujando.

4.3. Por qué el runner *tira* en vez de que el repartidor *empuje*

Tu enunciado sugería agente → tablero. En kanban eso es exacto para **crear** tarjetas. Para **ejecutarlas** en la máquina remota, el mismo principio manda al revés de lo que parece: los obreros reclaman trabajo cuando están libres y capacitados, nadie se lo asigna.

Tiene tres ventajas concretas en tu caso:

El servicio de Athena es loopback y su token es efímero. El README lo dice: no publicar en LAN, y el token se acuña al arrancar y se anuncia en la línea `ATHENA_SERVICE_READY`. Un runner que vive en el mismo PC lo captura y lo usa localmente. El PC-IA nunca necesita alcanzar el 8770 ni conocer ese token.

No hay que abrir puertos en el PC de trabajo. Solo salidas.

Es la semántica correcta: si el PC principal está apagado, sus tarjetas se quedan quietas en `pending/` sin costar un céntimo — que es exactamente lo que el estándar quiere que pase cuando el sistema no puede hacer algo.

El emparejamiento sigue siendo **central y ciego**: el repartidor decide qué perfil atiende cada tarjeta y anota el arnés. El runner solo reclama las que ya están casadas con un arnés que él aloja. Determinismo intacto.

5. Contratos concretos

5.1. OAuth: qué implementar exactamente

Grant: `client_credentials` (RFC 6749 §4.4). Agora es a la vez servidor de autorización y servidor de recursos. Es un fichero, no un producto.

```

http
POST /oauth/token
Content-Type: application/x-www-form-urlencoded

grant_type=client_credentials
&client_assertion_type=urn:ietf:params:oauth:client-assertion-type:jwt-bearer
&client_assertion=<JWT firmado con la clave privada del cliente>
&scope=cards:write cards:read

```

Recomendación fuerte: `private_key_jwt` (**RFC 7523**) en vez de `client_secret`. Cada cliente (Athena, ChatyGPT, el runner, el cron) tiene un par de claves; Agora guarda solo las públicas. No hay secreto compartido que robar de un fichero de configuración, y revocar un cliente es borrar una clave pública. Si prefieres empezar simple, `client_secret_basic` con secretos en el Credential Manager de Windows también vale y se migra después.

Access token: JWT RS256, TTL 600 s, `aud=agora`, `sub=<client_id>`, `scope`.

Scopes:

Scope	Quién lo tiene	Qué permite
<code>cards:write</code>	Athena, ChatyGPT, Telegram, cron	crear tarjetas
<code>cards:read</code>	todos	leer tablero y tarjetas
<code>board:claim</code>	solo los runners	reclamar, reportar progreso, cerrar
<code>board:admin</code>	solo tú	desbloquear, editar perfiles, forzar reparto

El detalle que hace que esto valga la pena. El estándar exige que el repartidor valide el `origin` de cada tarjeta contra una lista de confianza. Si haces que `origin` **se rellene desde el `sub` del token y el cliente no pueda escribirlo**, esa lista de confianza deja de ser una cadena en un YAML y pasa a estar firmada criptográficamente. Es la primera capa de defensa contra inyección, gratis, y es un argumento sólido para meter OAuth aquí y no en el broker.

El broker se queda con `x-admin-token`. No lo toques. Es un sistema estable y probado, su único cliente nuevo (Agora) está en la misma máquina y en la misma LAN, y añadirle un servidor de autorización no compra nada. La frontera OAuth es la que mira hacia tus aplicaciones; la frontera de token es la interna.

5.2. API del tablero

	
POST /oauth/token	→ token
POST /v1/cards	cards:write crear tarjeta (origin del token)
GET /v1/cards?state=&function=&flow=	cards:read censo
GET /v1/cards/{id}	cards:read tarjeta completa (front matter + cuerpo + Record)
GET /v1/board	cards:read vista agregada por columna
GET /v1/profiles	cards:read censo de perfiles con handles/refuses
GET /v1/events	cards:read SSE: card.created/claimed/progress/closed/blocked
POST /v1/cards/{id}/claim	board:claim reclamo con compare-and-swap; devuelve perfil renderizado
POST /v1/cards/{id}/progress	board:claim hito en el Record (obligatorio tras admisión)
POST /v1/cards/{id}/close	board:claim paths + línea de cierre
POST /v1/cards/{id}/yield	board:claim director: vuelve a pending SIN incrementar attempts
POST /v1/cards/{id}/unblock	board:admin
GET /health	—

`POST /v1/cards` acepta `Idempotency-Key`, igual que Athena. Dos veces la misma tarjeta no son dos tarjetas.

5.3. Traducción perfil + tarjeta → tarea del broker

Esto es el corazón del arnés `broker`. Nada aquí es negociable con el modelo: lo compone el repartidor con reglas.

	
json	

```
{
  "idempotency_key": "card:2026-08-30-transcribir-podcast-42:0",
  "content": {
    "prompt": "<PROFILE.md renderizado> + <SKILL.md montadas> + <cuerpo de la tarjeta> + <entradas por referencia>",
    "attachments": [{ "type": "broker_file", "metadata": { "file_id": "file_abc123" } }]
  },
  "output": { "format": "markdown", "language": "es" },
  "generation": { "temperature": 0.0, "seed": 20260830 },
  "model_requirements": { "max_cost_usd": 0.05 },
  "execution": { "strategy": "single", "timeout_seconds": 1200 },
  "risk": { "data_classification": "confidential" },
  "prompt_compression": "off",
  "priority": 100
}
```

De dónde sale cada campo:

Campo del broker	Lo decide	Nota
<code>idempotency_key</code>	repartidor	<code>card:<slug>:<attempts></code> . Un redespacho del mismo intento devuelve la tarea original (<code>200</code>) en vez de pagarla dos veces; un reintento real lleva otro <code>attempts</code> y sí es una tarea nueva.
<code>content.prompt</code>	perfil + tarjeta	el contrato primero, el encargo después. Nunca al revés.
<code>attachments</code>	tarjeta (<code>inputs</code>)	el repartidor sube los ficheros a <code>/api/v1/files</code> antes de despachar y espera a <code>ready</code> .
<code>execution.strategy</code>	perfil (<code>model_capacity</code>)	ver §5.5.
<code>risk.data_classification</code>	perfil	<code>confidential</code> por defecto para contenido de bóveda. El broker deriva <code>cloud_allowed</code> y <code>allowed_providers</code> solo (contrato ≥2.6): no mandes los tres.
<code>prompt_compression</code>	siempre <code>"off"</code>	ver trampa nº 2 en §7.
<code>generation.seed</code> + <code>temperature: 0</code>	perfil (<code>determinism: strict</code>)	ver §5.5.
<code>priority</code>	tarjeta (<code>priority: urgent</code> → 900)	

Y a la vuelta, el repartidor:

Lee `result` y, si hay, `GET /api/v1/tasks/{id}/artifacts`.

Escribe la salida en el destino (`destination` de la tarjeta, o el que infiera de la bóveda).

Lee `GET /api/v1/tasks/{id}/invocations` para saber **qué modelo concreto** la hizo y su coste, y lo escribe en el campo `model` de la tarjeta y en el `Record`. Esto es lo que te da el mapa de calor de gasto por tarea y por flujo que pide el estándar.

Mueve la tarjeta a `done/` con `paths` rellenos.

5.4. Traducción perfil + tarjeta → run de Athena

Para obreros de la segunda clase, el `agora-runner` en el PC principal:

```
□
http
```

```
POST http://127.0.0.1:8770/v1/runs
Authorization: Bearer <token efímero del servicio Athena>
Idempotency-Key: card:<slug>;<attempts>

{
  "objective": "<contrato del perfil + encargo de la tarjeta>",
  "workspace": "<destination o raíz de la bóveda>",
  "profile": "documents",
  "writes": "allow",
  "exec": "off",
  "deliverables": [ "<paths esperados>" ],
  "max_iterations": 12,
  "session_timeout_seconds": 1800
}
```

Después se suscribe a `GET /v1/runs/{id}/events` (SSE) y traduce eventos a hitos del `Record` de la tarjeta. Detalles que importan:

`writes` y `exec` **los declara el perfil, no la tarjeta**. Una tarjeta que pida más permisos de los que el perfil concede se bloquea por sospechosa. Esto es «la tarjeta pide, no manda» aplicado a capacidades, y encaja limpio con ADR-011 y ADR-014 (las extensiones restringen, nunca amplían).

`ask` **no sirve en modo tarjeta**. No hay interlocutor. Un perfil atómico usa `off` o `allow`, nunca `ask`: si no, el run se cuelga esperando una aprobación que nadie va a dar, y acabas con un zombi. El repartidor debe rechazar en el censo cualquier perfil de arnés `athena` que declare `ask`.

`deliverables` convierte la verificación por artefactos en verificación de *lo que se pidió* en vez de *lo que el run dice haber escrito*. Rellénalos siempre desde `destination`.

Un run que acabe en `verification_inconclusive` (ADR-027) **no es un cierre**: es una excepción legítima que va al `Record` y devuelve la tarjeta a `pending/` o la bloquea, según el `failure_policy` del perfil. Nunca un `done/` con `paths` vacío — eso es un cierre falso y contamina a los directores.

5.5. `models.yml`: capacidad declarada → parámetros del broker

El perfil declara nivel, nunca modelo. La tabla traduce:

model_capacity		execution.strategy	data_classification	max_cost_usd	Determinismo
minimum	single		local_only	—	target_model fijo + seed + temperature: 0
standard	single		confidential	0.05	enrutado; se registra el modelo usado
advanced	mixture_of_agents	(preset slow) o agent internal		0.50	enrutado; se registra

model_modality	Efecto
text	por defecto
audio	el fichero entra por <code>/api/v1/files</code> ; lo transcribe faster-whisper antes del despacho
image	el broker exige visión y descarta candidatos solo-texto; sin ninguno falla con <code>VISION_MODEL_UNAVAILABLE</code>

Cambiar de proveedor o irte a local sigue siendo editar esta tabla, que es la promesa del estándar y que tu broker ya cumple estructuralmente: no hay ni un modelo escrito a fuego en ningún perfil.

La tensión que tienes que resolver conscientemente. El estándar quiere «mismo perfil + misma tarjeta = mismo resultado». Tu broker enruta por evidencia: el mismo prompt puede ir a modelos distintos días distintos, y esa es precisamente su mejor característica. No son compatibles al 100 %. La salida es declararlo por perfil:

`determinism: strict` → `target_model` exacto, `fallback_allowed: false`, `seed`, `temperature: 0`. Para clasificadores, extractores y todo lo que alimente a otro paso. Reproducible y depurable.

`determinism: routed` → deja enrutar al broker y **anota el modelo real en la tarjeta**. Para redacción, resumen, análisis. Pierdes reproducibilidad bit a bit y ganas el enrutado adaptativo; el `Record` te dice siempre quién lo hizo.

6. Decisiones de colocación

6.1. El tablero vive en el PC-IA, y se reclama por API

El estándar apoya la reclamación en que mover un fichero es atómico. **Eso deja de ser cierto en cuanto el tablero se comparte entre máquinas por sincronización** (Syncthing, OneDrive, Dropbox): la sincronización es consistente *a la larga*, y dos máquinas pueden «mover» la misma tarjeta y descubrirlo diez segundos después. Por SMB el `rename` sí es atómico en el servidor y falla si el destino existe —que es justo el mecanismo—, pero pasas a depender de que la red esté sana en cada latido.

Recomendación: **un solo dueño del tablero** (el proceso de Agora en el PC-IA), tablero en disco local, y el reclamo remoto por API con compare-and-swap. Por dentro sigue siendo mover ficheros entre `pending/` e `in-progress/`, con lo que conservas la propiedad buena: los ficheros de texto siguen siendo el estado, se siguen pudiendo ver «con esos ojitos», y el `git` del tablero sigue siendo la auditoría. Lo único que cambia es que el reclamo tiene un árbitro en vez de un truco del sistema de ficheros.

Cuando en el futuro añadas un segundo VPS o un Mac Mini, se conecta como otro runner. La escalabilidad horizontal del estándar la conservas entera; lo que no haces es montar un sistema distribuido sobre una carpeta sincronizada, que es donde esto se rompe de verdad.

6.2. La bóveda: decide quién escribe

Es la pregunta abierta más importante y depende de dónde tengas el Cerebro Digital. Dos formas sanas:

Bóveda en el PC principal, compartida por SMB. Los obreros de inferencia del PC-IA escriben por ruta UNC. Simple, y la bóveda vive donde la lees a diario. Riesgo: un corte de red convierte un cierre en un fallo. Mitigable escribiendo primero a un `outbox/` local y moviendo después.

Un oficio `archivador` en el PC principal. Los obreros dejan su salida en `outbox/` del PC-IA y una tarjeta encadenada la coloca en la bóveda. Más fiel al estándar (un solo escritor por superficie, integración por tablero) y más piezas.

Empieza por la primera; si el enlace demuestra ser flojo, la segunda es una tarjeta más, no un rediseño.

Lo que **no** debe pasar, y el estándar es explícito: el código de Agora no vive dentro de la bóveda. Repo propio, como el broker.

6.3. Dónde viven los perfiles

`AGENTS/` en su propio repositorio git, en el PC-IA, leído por el repartidor en cada ronda (censo directo, nunca lista cacheada). Añadir un agente es copiar una carpeta y no avisar a nadie. Los perfiles son contratos versionados: cada cicatriz sube la versión.

7. Trampas concretas de esta integración

Las he sacado leyendo tu configuración real. Todas tienen solución barata, pero ninguna se ve venir.

1. `max_active_workflows: 1` es un invariante del broker, no una preferencia. Tu `capabilities` lo documenta así. Diez obreros atómicos despachados a la vez se serializan en el broker. Consecuencias: el `max_dispatches_per_round` de Agora debe ser bajo (2–3) para no llenar la cola del broker de tarjetas que van a esperar de todas formas, y las tarjetas urgentes deben viajar con `priority` alta para que la cola del broker las adelante. Si necesitas paralelismo real, va por perfiles con `data_classification: internal` que salgan a cloud (no compiten por VRAM), no por subir ese número.

2. Tienes `prompt_compression: aggressive` global. El compresor poda cortesías y relleno del español. Un `PROFILE.md` renderizado es prosa: sus garantías, su política de entrada mala y su ritual de cierre son exactamente lo que un compresor agresivo considera relleno. **Manda siempre `prompt_compression: "off"` en las tarjetas de obrero.** El contrato es la carga útil, no el envoltorio.

3. El umbral de zombi tiene que ser mayor que el timeout del broker. Tienes `default_task_timeout_seconds: 2400` y techo 3000. Un umbral de zombi de 30 minutos rescataría tarjetas cuya tarea del broker sigue viva, y pagarías dos veces. Umbral ≥ 60 min, y al rescatar, **cancela la tarea del broker** (`DELETE /api/v1/tasks/{id}`) antes de devolver la tarjeta a `pending/`. Para eso hay que guardar `broker_task_id` en el front matter de la tarjeta al reclamar.

4. `max_task_attempts: 3` en el broker y `max_attempts` en el tablero son dos contadores distintos. Si no los alineas, una tarjeta con 3 intentos puede consumir 9 ejecuciones del broker. Documenta cuál manda: el del tablero, y el del broker que sea el interno de un solo despacho.

5. `exclude_from_model_learning`. Déjalo en `false` para las tarjetas de producción: quieres que el router aprenda de ellas. Ponlo a `true` solo en las tarjetas de los `examples/case-*` cuando ejecutes la batería de regresión de un perfil, para no envenenar las métricas con prompts deliberadamente difíciles.

6. El `Record` necesita el modelo real y solo lo sabe `invocations`. El objeto de la tarea no siempre te dice qué modelo respondió tras un reintento o una síntesis. Una llamada extra a `/api/v1/tasks/{id}/invocations` al cerrar te da modelo, parámetros efectivos y coste. Sin eso, el campo `model` de la tarjeta es una suposición y el mapa de gasto no vale.

7. **Ficheros adjuntos: hay que esperar a `ready`**. Adjuntar un `file_id` que aún está en `converting` devuelve `409 ATTACHED_FILE_NOT_READY`. El repartidor debe subir y esperar antes de reclamar la tarjeta, o mejor: tratar «entrada declarada que aún no está lista» igual que trata el estándar una entrada inexistente — **espera, no incidencia**, la tarjeta se queda en `pending/` sin incrementar `attempts`. Es exactamente la puerta de entrada que el estándar ya describe.

8. **Colisión de nombres, otra vez**. El broker también tiene «skills» (`web_search`, `fetch_url`, `calculator`, `run_code`) y Athena tiene `SkillManifest`. La habilidad atómica (`transcribir-audio`) es un `SKILL.md` de saber hacer; las del broker son herramientas. En la documentación de Agora llámalas siempre **habilidades** (atómicas) y **herramientas** (broker/Athena), o dentro de un año no sabrás cuál es cuál al leer un perfil.

9. **«Dos puertas, un cuerpo de conocimiento» te sale casi gratis**. El `SkillManifest` de Athena (nombre, descripción, tareas aplicables, toolsets requeridos, instrucciones) es prácticamente el `SKILL.md` del estándar. Si haces que Agora y Athena lean **el mismo fichero**, la puerta conversacional y la puerta del tablero comparten las cicatrices de verdad, que es la invariante nº 2 de la sección «dos puertas» del estándar. Merece la pena forzar el formato para que coincidan desde el día uno; converger después es mucho más caro.

10. **El repartidor no puede tener modelo, y es tentador que lo tenga**. El día que una tarjeta no case con nadie vas a querer «que la IA decida». No lo hagas: es lo que destruye la reproducibilidad y lo que convierte un tablero depurable en una caja negra. Una tarjeta parada en `pending/` te está diciendo que a un perfil le falta un descriptor o que acabas de descubrir el siguiente agente que tienes que escribir.

8. Plan de implementación por fases

Cada fase termina con algo que funciona y con un criterio que se puede comprobar. No pases de fase sin él.

F0 · El tablero mínimo (sin API, sin OAuth, sin red)

Solo PC-IA. Un perfil real, el más aburrido que tengas: `transcriptor` o `resumidor`.

`cards.py` — parseo/serialización de front matter con la invariante de integridad: ninguna escritura puede dejar el front matter ilegible.

`Record` siempre en el cuerpo, nunca como campo YAML.

`board.py` — las seis carpetas, reclamo por `rename`, rescate de zombis.

`profiles.py` — censo directo de `AGENTS/*/PROFILE.md` en cada ronda.

`matching.py` — el embudo: `function` igual → `refuses` elimina → descriptor de `handles` más largo gana → `recipient` salta el embudo.

Función pura.

`dispatcher.py` — el latido, con `--dry-run` que reporta sin reclamar ni escribir nada.

`harnesses/broker.py` — la traducción de §5.3, con espera de `ready` en adjuntos y lectura de `invocations` al cerrar.

Criterio de aceptación: dejas una tarjeta en `pending/` a mano, te vas, y a los cinco minutos hay un fichero en la bóveda, la tarjeta está en `done/` con `paths` y `model` rellenos, y el `Record` cuenta lo que pasó en frases que entiendes sin abrir un log.

Prueba de la parte cara: el comprobador de solapamiento (`agora check-overlap`) que simula cada descriptor de `handles` de un perfil nuevo contra todos los demás y falla si alguno resuelve en otro sitio. Hazlo en F0 aunque solo tengas dos perfiles; en F4 ya es tarde.

F1 · Lectura por API

FastAPI, mismo stack que el broker. `GET /v1/cards`, `/v1/board`, `/v1/profiles`, `/health`, SSE en `/v1/events`. Sin escritura todavía, sin OAuth (token estático).

Criterio: ves el tablero desde el navegador del PC principal.

F2 · OAuth y creación de tarjetas

`POST /oauth/token` con `client_credentials`, JWT RS256, scopes. `POST /v1/cards` con `origin` derivado del `sub` del token e `Idempotency-Key`.

Primer cliente: un script suelto. Segundo cliente: Athena, con un `AgoraClient` **fuera del core** (`athena.adapters.agora`), en la misma

disciplina que ADR-002.

Criterio: desde una sesión de Athena le pides «déjame una tarjeta para transcribir esto» y aparece en `pending/` con `origin: athena`.

F3 • Runner remoto y arnés Athena

`agora-runner` en el PC principal: se autentica, sondea `GET /v1/cards?state=pending&harness=athena`, reclama con CAS, lanza `POST /v1/runs` en el servicio local de Athena, traduce el SSE a hitos del `Record`, cierra la tarjeta.

Criterio: una tarjeta que necesita escribir en un repositorio del PC principal se ejecuta con verificación de Athena y cierra con `deliverables` comprobados. Y con el PC principal apagado, esa misma tarjeta se queda quieta en `pending/` sin coste ni error.

F4 • Directores

La regla de oro (un hijo cada vez), el `yield` sin incrementar `attempts`, la puerta de entrada por `dependent_task`, y el **autochequeo de cableado**: antes de morir, el director relee su propia tarjeta y verifica que la referencia coincide exactamente con el nombre del fichero del hijo. Sin esa regla el director está incompleto. Más `KANBAN/scheduled/` con materialización idempotente por nombre de fichero.

Criterio: un flujo de tres pasos (transcribir → resumir → redactar) se completa solo; y si el paso 1 no produce nada, los pasos 2 y 3 nunca llegan a existir.

F5 • Explotación

Panel de coste por tarjeta y por flujo desde el campo `model` y el coste de `invocations`. Informe semanal. Segunda máquina si hace falta. El CEO solo cuando tengas dos directores reales funcionando.

9. Estructura de repositorio propuesta

Agora/	← repo propio, FUERA de la bóveda
— agora/	
— cards.py	parseo, integridad de front matter, Record
— board.py	seis carpetas, reclamo atómico, zombis
— profiles.py	censo + comprobador de solapamiento
— matching.py	el embudo determinista (puro, sin E/S)
— dispatcher.py	el latido; --dry-run
— scheduled.py	materialización de tarjetas periódicas
— capacity.py	models.yml: capacidad → parámetros del broker
— harnesses/	
— broker.py	obrero de inferencia
— athena.py	publica tarjeta reclamable por el runner
— cli.py	plantilla de subprocesso genérica
— api/	
— app.py	FastAPI
— oauth.py	token endpoint, JWT, scopes
— cards.py	CRUD + claim/progress/close/yield
— events.py	SSE
— security.py	origen desde el token, detector de tarjeta sospechosa
— config.py	
— agora_runner/	← se instala en el PC principal
— adapters.yml	arneses y sus plantillas
— models.yml	capacidad × arnés → modelo/parámetros
— agora_config.yaml	
— scripts/install_windows_service.ps1	
— tests/	
AGENTS/	← repo propio, contratos versionados
— transcriptor/	
— PROFILE.md	
— skills/transcribir-audio/SKILL.md	
— examples/case-01/{input.md,expected-output.md}	
ANBAN/	← disco local del PC-IA, git para auditoría
— pending/ in-progress/ done/ archive/YYYY-MM/ scheduled/ blocked/	

10. Lo que no hay que hacer

No pongas el tablero en el camino de la inferencia de Athena. Athena sigue hablando con el broker directamente, con su adaptador actual, para su bucle.

No metas un modelo en el repartidor. Nunca. Ni «solo para las tarjetas que no casan».

No compartas el tablero por sincronización de ficheros entre máquinas. Un dueño, reclamo por API.

No añadas OAuth al broker. La frontera OAuth mira hacia tus aplicaciones, no hacia dentro.

No construyas un director antes de tener dos obreros probados. Un director multiplica el fallo, no lo arregla.

No uses `ask` en perfiles de arnés Athena. En modo tarjeta no hay interlocutor: es un zombi con pasos extra.

No fusiones `AthenaProfile` con `PROFILE.md`. Componen; no son lo mismo.

11. Decisiones que te quedan abiertas

¿Dónde vive la bóveda? De esto depende §6.2, y es lo único que puede cambiar la forma del despliegue.

¿Qué proporción de tus encargos necesita herramientas? Si es baja, F3 puede esperar mucho y el sistema vale la pena antes.

`determinism: strict` **por defecto** o `routed` **por defecto?** Mi recomendación: `strict` para todo lo que alimente a otro paso de un flujo, `routed` para todo lo que produzca la salida final que va a leer un humano.